Asteroid Hunter: Start Here

Everyone reads this first

Machine Learning (702DB0C015), B.Tech CSE (Data Science), Semester V, 2026-27

Contents

1. 01 Project Overview
2. 02 Domain Primer
3. 03 ML Primer
4. 04 Course Mapping
5. 05 Team Plan
6. 06 Whole Project Viva QA
7. 07 AI Use and Integrity
8. 08 Glossary
9. Diagrams

01. Project overview

The idea in plain words

Telescopes photograph the same patch of sky again and again. Stars stay put. **Asteroids move**, so over a few minutes an asteroid is a dot that creeps in a straight line. Surveys like the Zwicky Transient Facility (ZTF) take millions of images, and almost all the dots they flag are junk (bad image subtraction, cosmic rays, noise). Our project builds a pipeline that:

1. takes ZTF's **difference images** (tonight's picture minus an old picture of the same sky, so everything that did not change disappears),
2. finds every blob left over,
3. uses **classical machine learning** (29 measured numbers + PCA per blob, then XGBoost decision trees) to decide which blobs look like a real moving object (not junk),
4. **links** blobs from different frames into straight-line tracks,
5. uses a second model (**gradient-boosted trees**) to score each track,
6. compares the tracks with the catalogue of known asteroids, and keeps what is unknown for a person to examine.

It was inspired by an online post where someone found a planet candidate in old Kepler telescope data using AI-assisted code (we checked that post: the signal belongs to an already confirmed planet, see `project/docs/kepler_post_check.md`).

Why machine learning is needed here

- One night of one CCD quadrant (5 frames) gives about **36,000-41,000 blobs**, and only about **3% are real** (validation nights: 3.2%). Hand-written rules for "looks like a real object" fail because junk comes in many shapes (dipoles, spikes, edges, ghosts).
- A **classical ML model** learns those shapes from labelled examples: we measure 29 numbers per blob (is it star-shaped? was it already in the old picture? one hot pixel?) plus 20 PCA numbers, and XGBoost learns which combinations mean real. A rule-based filter could not. (We first used a deep-learning CNN; it scored the same, so the deployed system uses classical ML only and the CNN is kept as a comparison: `project/docs/classical_realbogus.md`.)
- The **tracklet classifier** combines about 17 numbers per track (speed, straightness, steady brightness, what the first model thought, ...) better than any single threshold.
- Everything physical (matched filtering, linking by straight lines, shift-and-stack) is *not* ML: it is search and statistics. Knowing which part needs learning and which does not is part of good ML design.

The pipeline (see `11_Diagrams/01_system_overview.svg`)

Stage	What happens	Owner
Data	ask the archive, download 4 files per exposure, mask bad pixels, estimate noise, detect blobs, split by night	Aaryan
Labels + ML model	cut 3x31x31 stamps, labels from planted fakes and known asteroids, normalise/balance, 29 features + PCA, XGBoost gives p(real); SVM/forest/CNN compared; k-means on junk	Paras
Linking + classifier	join blobs into tracks, 17 features, XGBoost, cross-match with known asteroids, metrics	Satvik
Evaluation + discovery engine	scoreboard, honest testing, shift-and-stack (deep search), fast movers, live viewer	Advik

Headline results (all from `project/reports/scoreboard.csv` or the lesson documents)

What	Number
Our blob model (XGBoost, 29 features + 20 PCA) vs the old 6-number baseline (validation nights, 67,041 detections, 3.2% real)	average precision **0.902** vs **0.783**

What	Number
False alarms per frame at 90% recall	**158** (our model) vs **806** (baseline): 5.1x fewer
Our model on the untouched test night	average precision **0.938**
Deep learning (CNN), for comparison only	0.898 validation, 0.944 test: a tie, so we deploy classical ML
Whole pipeline, v6 (classical) vs v4 (CNN) on the scoreboard	same known asteroids on every split (40/44 on test); depth 21.28 vs 21.34
Tracklet classifier	near-perfect average precision (about 0.999) on planted fakes, which is why we do not use it to judge progress
Depth: faintest brightness where 50% of planted fakes are found (test night 2019-02-08)	v1 single frames **20.5**, v4/v6 **21.3-21.4**, deep search **22.1** (bigger = fainter)
Real search of the test night	**31 known asteroids recovered** (brightness 16.2 to 21.8), 0 unexplained candidates above threshold
Fast (trailed) objects	97% of planted trails recovered on the nights used to choose the cuts; 93% on a later 10-frame night
3x more training data (v5)	did **not** help at equal false alarms

What we are careful NOT to claim

- **No discovery.** One faint object in a related two-night study passes every check we have (orbit fit, noise tests, no catalogue match). It still needs a human reading the images and the Minor Planet Center's process. Nothing has been reported as a discovery.
- The best depth number (22.1) is for one night and one CCD quadrant.
- Our "real" labels come partly from simulation (planted fakes). We checked that fakes look like real asteroids (a classifier cannot tell them apart: AUC 0.47) but that is a check on certain features, not a proof.

The story of how it got better (good for the viva)

1. **v1**: CNN + XGBoost on single frames. Looked great (AP 0.999 for tracklets). The scoreboard showed the truth: depth only ~20.5.
2. We **measured where asteroids were lost**: at linking, not at the classifier. Relaxing linking found 11% more asteroids but created ~1,500 chance tracks per night (these became the hard negatives).
3. A bigger multi-frame CNN **tied** the simple XGBoost model, so we kept the simple one (**v4**).
4. 3x more training data (**v5**) did **not** help: the limit is how many photons one 30-second frame holds.
5. Therefore we added the **deep search** (adding frames along a trial motion): +0.7 magnitude.
6. **Classical ML only (v6)**: we replaced the CNN with XGBoost on 29 measured features + PCA. It tied the CNN on blobs and on the whole scoreboard, trains in seconds, and every input has a meaning, so it is the deployed system; the CNN stays only as a comparison.
7. Along the way, real bugs were found by checking results against reality: SkyBoT catalogue positions were ~2.9 arcsec off; telescope pointing shifts between exposures; a feature that wrongly flagged 22% of real tracks. Each was fixed and a test written.

Where everything is

- Lessons (one per phase, with a quiz): `project/docs/*.md`; matching notebooks: `project/notebooks/*.ipynb`.
- Code: `project/src/asthunt/<stage>/<verb_object>.py` (one purpose per file). Tests: `project/tests/`.
- Settings: `project/configs/*.yaml`. Trained model versions: `project/models/v1 ... v5`.
- The plan and status table: `project/PLAN.md`.

02. Domain primer: the astronomy you need (no prior astronomy assumed)

You do not need to be an astronomer. You need about twelve ideas, each explained so you can say it in your own words in a viva.

1. What an asteroid is, and why it shows up as a moving dot

Asteroids are rocky bodies orbiting the Sun (most between Mars and Jupiter, the *main belt*). From Earth, stars are so far away that they appear fixed. An asteroid is close enough that it visibly **moves against the stars**: tens of arcseconds per hour for a typical main-belt asteroid near opposition (our data: 28-48 arcsec per hour). An arcsecond is 1/3600 of a degree. Over the ~30 minutes of 5 consecutive exposures, an asteroid moves in a **nearly straight line at a nearly constant speed**. That is the physical fact the whole *linking* stage relies on.

2. The telescope and the survey

ZTF (Zwicky Transient Facility) photographs the northern sky with a camera at Palomar Observatory (California). The camera has 16 CCD chips; each chip is read out in 4 **quadrants** of about 3072 x 3080 pixels. One pixel is about **1.01 arcseconds**. A standard exposure lasts **30 seconds**. Images come in filters (we use **r band**, "zr"). The data is public through **IRSA** (a NASA/IPAC archive).

A **unit** in our project = one *field* (a patch of sky) + one *CCD* + one *quadrant* + one *night*. Example: `000618_c03_q4_2019-02-08` = field 618, CCD 3, quadrant 4, night 8 Feb 2019.

3. The four files per exposure

File	What it is
science image	the picture itself (about 38 MB)
reference image	a deep picture of the same sky built from many old exposures (we recompute it as science minus difference)
difference image	science minus the (blurred, aligned) reference: static things cancel
mask	16 yes/no flags per pixel: saturated, dead, cosmic ray, ghost, ...
PSF	the blur shape of a point source in the difference image

4. Difference imaging: the key trick

`difference = science − reference`. Stars, galaxies and the sky glow are the same in both and cancel. What remains: new or moving things (**positive blobs**, what we want), faded things (negative blobs, ignored), imperfect cancellation of bright stars (**dipoles**), cosmic rays, and noise. Most blobs in a difference image are junk, which is exactly why a classifier is needed.

5. Noise, sigma, and signal-to-noise (SNR)

Every pixel has random noise. We measure its spread (**sigma**) robustly (with the *median absolute deviation*, MAD, so bright blobs do not fool it: sigma = 1.4826 x median(|x - median|)). A blob "5 sigma" bright is 5 times the noise level. **SNR** = measured flux divided by its uncertainty. With pure noise, about 1 pixel in 3.5 million exceeds +5 sigma, so a 5-sigma blob is unlikely to be chance, *unless you looked in many places* (see 12).

6. The PSF (point spread function)

A star or asteroid is a point of light, but the atmosphere and optics blur it into a small round blob (about 2 pixels wide for us). The blur shape is the **PSF**. Knowing it lets us do **matched filtering**: smooth the image with the blur shape, which makes faint point-like sources stand out of random noise.

7. Magnitudes (how astronomers measure brightness)

A **magnitude** is a log brightness scale where **bigger = fainter**. 5 magnitudes = a factor of 100 in brightness. So one magnitude is a factor of about 2.5. Our single-frame limit is about **magnitude 20.5-21.4**; the deep search reaches about **22.1**, which is 0.7 magnitudes fainter, a factor of about 1.9 in brightness. The faintest star the naked eye can see is about magnitude 6, so these asteroids are about a million times fainter (15 magnitudes = a factor of 10^6).

7b. Opposition

At **opposition** the asteroid is opposite the Sun in the sky: closest to Earth and brightest, and there are the most of them near the *ecliptic* (the plane of the planets). Our test night, 2019-02-08, was near opposition, so it is both the most asteroid-rich and a deliberate "distribution shift" from the training nights.

8. WCS (sky coordinates) and why pointing matters

The **WCS** converts pixel positions to sky positions (RA, Dec). The telescope does not point at exactly the same pixels in consecutive exposures (shifts up to 12 pixels). An asteroid moves in a straight line **on the sky**, not in each frame's pixels. So we put every blob on frame 1's common sky grid before linking. Forgetting this was our biggest bug.

9. Known versus unknown asteroids

Hundreds of thousands of asteroids are catalogued. The **Minor Planet Center (MPC)** keeps the official list. We look them up with **SkyBoT** (a service that predicts where known objects are) and **JPL Horizons** (NASA's very accurate ephemeris). We measured SkyBoT positions to be about 2.9 arcsec (2.7 px) off, while Horizons is within 0.26 px. Almost everything we detect is **already known**; only objects matching nothing are *candidates*.

10. Tracklet

A **tracklet** = a set of detections in different frames that line up as one moving object. It needs at least 3 of the 5 frames, a straight line within 1.5 pixels, and a speed between 8 and 120 arcsec/hour.

11. Injection-recovery and completeness

We cannot know which real asteroids are in an image, so we **plant fake asteroids** (known brightness, speed, position) into the real images, run the normal pipeline and count how many come out. The fraction recovered, as a function of brightness, is the **completeness**. "50% completeness at magnitude 20.5" means: half of the fakes of that brightness are found. This is the standard method in survey astronomy.

12. Look-elsewhere effect (why we need a 6-sigma cut in the deep search)

If you test thousands of places or motions, some random noise will reach 5 sigma by chance. In the deep search we try 4,888 motions at every pixel, so noise alone produced 6,179 peaks at 5 sigma but only 44 at 6 sigma (on a pure-noise test). So we require 6 sigma. This is the same logic as multiple-testing correction in statistics.

Two more terms you will hear

- **Dipole**: a bright half next to a dark half, the typical leftover of a bright star that did not subtract perfectly.
- **Ghost / bleed trail**: optical or detector artifacts (a bright star's charge spilling along a CCD column). Bleed trails line up along columns and can fool a fast-object search; we found six of seven fast "tracks" we inspected were such artifacts.

Self-test (say each in one sentence)

1. Why do we use difference images? 2. What does "5 sigma" mean? 3. Which is brighter, magnitude 18 or 21? By how much? 4. What is opposition and why did we choose such a night for the test?
2. Why must pointing shifts be corrected before linking? 6. What is a tracklet? 7. What does 50% completeness mean? 8. Why can an astronomer not just trust one 5-sigma blob?

03. ML primer: the ideas we used, tied to the syllabus

Each idea has three lines: **what it is**, **where it appears in our project**, **the syllabus session it matches** (from the course policy document). If you can explain these in your own words, you can pass a viva on any part.

A. The problem setting

Supervised learning: learn a function from examples with known answers. *Ours:* XGBoost learns real/bogus blobs from 49 measured numbers and real/bogus tracklets from 17 numbers. **Unsupervised learning** finds structure without labels. *Ours:* PCA compresses each stamp's pixels; k-means groups the junk blobs into 6 kinds. *Syllabus: types of learning, supervised vs unsupervised.* **Classification vs regression**: classification predicts a category (real/bogus), regression a number. *Ours:* classification; one small regression (a straight-line fit from stack SNR to magnitude in the deep search: mag = 26.25 - 5.23 log10(SNR)). *Syllabus: linear regression.* **Features and labels**: a feature is a number measurable on new unlabelled data; the label is the answer. *Ours:* `mag` and `true_rate` existed only for planted fakes, so they were illegal features (target leakage). *Syllabus: ML requirements (CO1).*

B. Models

Logistic regression: `p = sigmoid(w.x + b)`, trained by minimising log-loss. *Ours:* the baseline on 6 numbers per detection (AP 0.780), and in the model comparison on tracklets. *Syllabus: logistic regression, hypothesis, cost function.* **Gradient descent**: repeat `w <- w - learning_rate x gradient`. *Ours:* written by hand in numpy in `notebooks/neural_networks.ipynb`, then Adam in PyTorch for the CNN (kept as a comparison). *Syllabus: gradient descent, cost function.* **Neural network**: layers of "weighted sum, then non-linearity". Without the non-linearity, stacked layers collapse into one linear function. *Ours:* the from-scratch notebook and the CNN, which we keep only as a comparison: the deployed model is classical. *Syllabus: NN hypothesis, cost, activation functions.* **Backpropagation**: the chain rule applied from the output back, so one sweep gives every gradient. For sigmoid + cross-entropy the output error is simply `p - y`. *Ours:* coded by hand and checked numerically (matches to 8 decimals). *Syllabus: backpropagation.* **Convolutional neural network (CNN)**: filters (3x3 grids of learned weights) slide across an image looking for a local pattern; pooling shrinks the image so deeper filters see larger areas. *Ours:* 5 convolution layers over 3 stacked pictures; 72,145 weights; AP 0.898, a tie with our classical model (0.902), so comparison only. *Syllabus: neural networks (applied).* **Feature engineering**: turning raw data into numbers with a meaning. *Ours:* 29 numbers per stamp (star shape, "already in the old picture?", sharpness, dark hole next to it...). The most useful one: how much the difference picture looks like a star. **SVM (support vector machine)**: finds the widest margin between the classes; with an RBF kernel the boundary can curve. *Ours:* compared as a stage-1 model (AP 0.886 validation, 0.877 test). *Syllabus: SVM, relation to logistic regression.* **PCA**: rotates the data onto the directions of largest variance and keeps the first few. *Ours:* 675 pixels -> 20 numbers (78% of the variance), used as extra features. *Syllabus: PCA.* **k-means clustering**: places k centres and assigns each point to the nearest, repeating until stable. *Ours:* 6 clusters of junk: 93% faint noise, star leftovers, bright stars, old-picture-only. *Syllabus: clustering, k-means.* **Decision tree -> random forest -> gradient boosting**: a tree asks yes/no questions; a forest averages many random trees (bagging, lowers variance); boosting builds trees one after another, each correcting the errors so far. *Ours:* XGBoost on 49 numbers per blob (600 trees, depth 4, the deployed stage-1 model) and on 17 numbers per tracklet (300 trees, depth 3, learning rate 0.05). *Syllabus: trees, bagging, random forests, boosting.*

C. Making models behave

Overfitting: the model memorises noise; training error falls, validation error rises. *Ours:* a demo on 40 points (validation loss lowest at step 22 then climbs), and early stopping in the CNN. *Syllabus: bias and variance, evaluating a hypothesis.* **Regularisation**: anything that stops the model fitting noise: a penalty on large weights (L2 in logistic regression, strength `C`), **dropout** (0.3 in the CNN head), **early stopping** (patience 4), **subsampling** (XGBoost uses 80% of rows and columns per tree), **data augmentation** (random rotations and mirrors). *Syllabus: regularization (CO2).* **Feature scaling / normalisation**: put numbers on a comparable scale. *Ours:* per-stamp robust normalisation `(x - median) / (1.4826 MAD)`, clipped to [-10, 50]. Trees need no scaling. **Class imbalance**: real objects are about 3% of detections. We balanced *training* (all real + 5 bogus

per real = 16.7% real) but kept *validation* realistic. Side effect: raw probabilities are inflated; fix with base-rate adjustment or isotonic calibration.

D. Evaluating honestly (the heart of "model evaluation", worth 3 marks)

Train / validation / test: train fits the model; validation chooses settings and thresholds; test is opened once. *Ours:* **split by night**, decided in a config file before modelling. *Syllabus: train-validation-test.* **Data leakage**: information at training time that will not exist at use time. Four forms we found: target leakage (`mag`), group leakage (all tracklets from 4 images), shortcut learning (random bogus tracklets 222+ px off a line), distribution shift (test night sharper). Evidence: accuracy 100% -> 50% -> 99% depending on the test data. **Cross-validation**: repeat train/test over several splits. *Ours:* `GroupKFold` by night (never random rows). **Cross-fitting** is the same idea used so a second model does not see over-confident first-model scores. **Confusion matrix, precision, recall, false-positive rate**: see `04_Satvik/07_Diagrams/04_confusion_metrics.svg` . *Syllabus: error metrics for skewed classes, precision/recall trade-off.* **Accuracy paradox**: "always bogus" scores 96.8% accuracy and finds nothing. With rare positives use recall, precision and the **precision-recall curve (average precision)**. ROC AUC looked excellent (0.95) when precision at useful recall was under 20%. *Syllabus: ROC curves.* **Threshold / operating point**: pick the cut on validation to keep 90% of real objects, then count false alarms. **Calibration**: do scores mean probabilities? (reliability plot; isotonic regression; Bayes base-rate correction). **Error analysis**: look at the model's most confident mistakes. *Ours:* the CNN's top "bogus" scores (in the first version) were known asteroids -> exposed a 2.9 arcsec catalogue offset (label error). 9 "false positives" on the test night were real known asteroids our labels missed. *Syllabus: error analysis.* **Completeness curve (injection-recovery)**: the domain's version of a recall-by-condition plot (recall vs brightness). **Adversarial validation**: train a classifier to tell fakes from real; AUC 0.47 means it cannot -> fakes are realistic. **Fair comparison**: compare models at the same false-alarm level, not at each one's own threshold.

E. Ideas from the research side

Stacking: one model's output is another's input (stage-1 scores are features for the tracklet XGBoost). **Ensembling**: averaging the two models gave only a tie. **Occam's rule**: when models tie, use the simplest (the CNN and a multi-frame CNN both tied XGBoost; we deploy XGBoost). **Negative results are results**: 3x more data did not help; the limit was physics (photons in one frame), not model size. **Look-elsewhere effect / multiple testing**: pick the best of 4,888 trials and noise looks significant. Calibrate on pure noise (6 sigma), and run a **null test** (shuffle the frame times; real movers stop lining up; we got 0 false tracks).

F. Syllabus topics we did NOT use (be honest about them)

Linear regression as a main model, polynomial regression, the normal equation, hierarchical clustering, regression trees. (SVM, PCA and k-means are now used: section B.) See `04_Course_Mapping.md` for how to cover them with small extra experiments if your faculty expects them. We touch **anomaly detection** only in spirit: "tracks that match nothing in the catalogue" is an anomaly filter, but it is a catalogue lookup, not a learned anomaly model.

Self-test

1. Why does a network need a non-linear activation? 2. What does `p − y` have to do with backprop? 3. Why is accuracy useless for 3% positives?
2. What is the difference between bagging and boosting? 5. Why split by night instead of randomly? 6. What is cross-fitting and what goes wrong without it?
3. Why might a model with a worse depth on the scoreboard be just as good as another? 8. What is the look-elsewhere effect?

04. The course policy document and what we have done against it

Source: *Course Policy, Machine Learning (702DB0C015)*, B.Tech CSE (Data Science), Semester V, 2026-27, MPSTME NMIMS. Coordinator: Prof. Abhay Kolhe.

1. What the document asks of the project

- **Weight:** the project is 15 marks of the 50-mark internal assessment (marked at the 4th and 11th week).
- **Team size:** "Group of 2-3 students". **We are four.** Ask the coordinator to approve a group of four (or decide how to present it) *before* the presentation.
- **Topic approval:** "the faculty member should approve the topic and title of the project by the end of 3rd week". Confirm that approval exists in writing; if it does not, get it now.
- **What to do:** "Identify and implement a journal/IEEE conference research paper based on some topic relevant to the course... Read, understand, summarize and implement the paper(s)."
- **Marking scheme:** Problem Understanding and Formulation (3), Data Preprocessing and Exploration (3), Model Development (4), Model Evaluation and Performance (3), Documentation and Presentation (2).
- **Course outcomes the project maps to:** CO1 (describe requirements of ML algorithms), CO2 (select and regularise linear and nonlinear models), CO3 (design a machine-learning system), CO4 (classification and clustering to extract insight).
- **GenAI policy (read `07_AI_Use_and_Integrity.md`):** AI may help with planning, research, learning new concepts and deployment, but students must not use it "to develop entire project; the core implementation and critical thinking aspects of the project must be the students' own work", and "students need to cite the references used".

2. Which research paper(s) does this project implement?

The document requires a paper. Our project was inspired by a post, not by a single paper, so we map it to the closest published work. **Confirm the final choice with your coordinator.** All four were checked to exist:

Role	Paper	How our project relates
Primary (CNN part)	Duev, D. A. et al. (2019), *Real-bogus classification for the Zwicky Transient Facility using deep learning*, MNRAS 489, 3582 (arXiv:1907.11259)	Same task family: a CNN that separates real detections from false positives in ZTF data. Differences to state openly: ours scores moving-object blobs, uses 31x31 pixel science/reference/difference stamps, and gets labels from planted fakes plus known asteroids. Read the paper and write down exactly what you replicated and what you changed.
Supporting (ML in ZTF)	Mahabal, A. et al. (2019), *Machine Learning for the Zwicky Transient Facility*, PASP 131, 038002 (arXiv:1902.01936)	Overview of ML at ZTF: real/bogus, star/galaxy, asteroid detection, training samples. Good for the problem formulation and related work.
Supporting (deep search)	Whidden, P. J. et al. (2019), *Fast algorithms for slow moving asteroids: constraints on the distribution of Kuiper Belt Objects*, AJ 157, 119 (arXiv:1901.02492)	GPU shift-and-stack over a huge number of trial motions (their KBMOD software). Our deep search is the same idea at a smaller scale (4,888 motions).
Supporting (data)	Masci, F. J. et al. (2019), *The Zwicky Transient Facility: Data Processing, Products, and Archive*, PASP 131, 018003 (arXiv:1902.01872)	Describes the difference images and the archive we use.
Inspiration	Shallue, C. J. & Vanderburg, A. (2018), *Identifying exoplanets with deep learning...*, AJ 155, 94 (arXiv:1712.05044)	The "AI on old Kepler data" idea the project started from.

Role	Paper	How our project relates
Methods	Chen, T. & Guestrin, C. (2016), *XGBoost: a scalable tree boosting system*, KDD; Kingma, D. & Ba, J. (2015), *Adam: a method for stochastic optimization*, ICLR	The boosting library and the optimiser we use.

(Only the abstracts and bibliographic details were verified online; you must read the papers yourselves before citing details.)

3. The marking scheme against what we have (evidence and owner)

Criterion (marks)	What we have	Evidence in the folder	Owner
Problem understanding and formulation (3)	The problem, why ML, why difference images, success metrics (recall at a fixed false-alarm level, depth), what is out of scope	`01_Project_Overview`, `02_Domain_Primer`, `project/docs/astronomy_data_primer.md`, `project/PLAN.md`	**Aaryan**
Data preprocessing and exploration (3)	Metadata-first acquisition, split by night, masks, noise estimation, EDA (thresholds, SNR histograms, stamp zoo), leakage audit, normalisation, augmentation, balancing	`project/docs/data_acquisition.md`, `difference_imaging.md`, `leakage_audit.md`, `dataset_building.md` + notebooks	**Aaryan** (data) and **Paras** (dataset building)
Model development (4)	CNN from scratch (backprop notebook first), XGBoost with explained settings, logistic-regression baseline, model comparison (tree, forest, boosting), a multi-frame model that tied	`project/docs/realbogus_cnn.md`, `tracklet_classifier.md`, `multiframe_model.md`, `ml_basics.md`, `neural_networks.md`	**Paras** (CNN) and **Satvik** (classical models)
Model evaluation and performance (3)	Metrics for skewed classes, PR vs ROC, calibration, completeness curves, scoreboard on held-out nights, comparison at equal false alarms, negative results	`project/docs/metrics_evaluation.md`, `scoreboard.md`, `more_data.md`, `reports/scoreboard.csv`	**Advik** (with Satvik on metrics)
Documentation and presentation (2)	One lesson per phase with a quiz, notebooks, architecture diagrams, this folder, live viewer demo	`project/docs/`, `project/docs/architecture/`, every `03_Presentation_Guide.md`	**Advik** (lead) and everyone

4. Syllabus coverage, honestly

"Used" = the topic is really in the project (with the file). "Partly" = touched but not a main method. "Not used" = a gap.

Syllabus topic (session)	Status	Where / note
Supervised vs unsupervised learning	**Used**	supervised: XGBoost, SVM, forest, logistic regression; unsupervised: PCA and k-means (`docs/classical_realbogus.md`)
Linear regression, multiple, polynomial	Partly	one linear fit (magnitude vs log SNR) in the deep search; no regression model as such
Logistic regression	**Used**	baseline (`models/train_baseline_classifier.py`, `docs/ml_basics.md`)

Syllabus topic (session)	Status	Where / note
Hypothesis, gradient descent, cost function	**Used**	from-scratch numpy network in `notebooks/neural_networks.ipynb`
Normal equation	Not used	
Regularisation	**Used**	L2 in logistic regression, dropout, early stopping, XGBoost subsampling, augmentation
Neural networks: hypothesis, cost, backprop, activation	Comparison only	`docs/neural_networks.md` ; the CNN is kept as a comparison, the deployed model is classical
Evaluating a hypothesis, train/val/test	**Used**	split by night; leakage audit
Bias and variance curves	Partly	overfitting demo with train/validation loss; no formal learning-curve study
Error analysis, metrics for skewed classes, precision/recall trade-off	**Used**	`docs/metrics_evaluation.md` , error analysis in the CNN and tracklet docs
Decision trees, classification trees, trees vs linear models	**Used**	`docs/tracklet_classifier.md` (depth-3 tree vs linear)
Regression trees	Not used	
Bagging, random forests, boosting	**Used**	forest and XGBoost cross-validated
SVM (maximal margin, SVC, multiclass, relation to logistic regression)	**Used**	RBF SVM in the stage-1 model comparison (`scripts/compare_realbogus_models.py`)
ROC curves	**Used**	ROC vs precision-recall
PCA, clustering, k-means, hierarchical clustering	**Used** (not hierarchical)	PCA features in the deployed model; k-means on the junk blobs (`reports/junk_clusters.png`)
Anomaly detection and large-scale ML	Partly	"unknown tracks" is an anomaly-style filter by catalogue lookup; GPU shift-and-stack, float16 stamps, streaming pipeline for scale

Gaps closed by the classical-ML version (v6)

1. **SVM** compared with logistic regression, random forest and XGBoost as the stage-1 model (CO3).
2. **PCA** as features of the deployed model and **k-means** on the junk blobs (the "bogus zoo": faint noise, star leftovers, bright stars, old-picture-only): classification **and** clustering (CO4).
3. A **learning curve** (validation score vs number of training nights): covers bias/variance curves. Each is a day of work for one person and is a *new* experiment you must run and understand yourselves. Ask Advik if you want the starting code reviewed.

5. What we have done, in one paragraph (for the report's abstract)

We implemented an end-to-end machine-learning pipeline for detecting moving objects in archival ZTF difference images. We built reproducible data acquisition split by night, generated labels by injection-recovery and catalogue matching and audited their quality, trained a classical real/bogus classifier, XGBoost on 29 measured stamp features plus 20 PCA components (average precision 0.902 on validation, 0.938 on the test night, 5.1x fewer false alarms than a logistic-regression baseline), compared it with logistic regression, an SVM, a random forest and, as a reference, a CNN (which tied it), clustered the junk with k-means, linked detections into tracks and scored them with gradient-boosted trees, and evaluated the whole system with a scoreboard on held-out nights using planted asteroids (depth of about 20.5 magnitudes for single frames, 21.3 after improving linking, 22.1 with a shift-and-stack deep search). We report negative results honestly (deep learning did not beat classical ML; three times more training data did not help) and make no discovery claim.

6. Things to decide with your coordinator this week

- [ ] Is a group of four acceptable, and how will individual contribution be assessed?
- [ ] Is the topic approved (title, paper choice)? Get it in writing.
- [ ] How should AI assistance be declared (see `07_AI_Use_and_Integrity.md`)?
- [ ] Is a live demo allowed/expected? (The live viewer needs the data on a laptop.)

- [x] SVM, PCA and clustering are now covered (classical-ML version, v6).

05. Team plan: who owns what

The split (by pipeline stage, matched to the marking scheme)

	Owner	One-line title	Rubric marks it carries
1	**Advik** (lead)	**System architecture, evaluation, deep search, live viewer**	Model evaluation (3), documentation and presentation (2), plus the integration of everyone's work
2	**Aaryan**	**Problem, domain, data and preprocessing**	Problem understanding (3), data preprocessing and exploration (3, shared)
3	**Paras**	**Labels, dataset building and the classical ML model (features, PCA, XGBoost; SVM/forest/CNN compared; k-means)**	Model development (4, shared), data preparation (shared)
4	**Satvik**	**Linking, the tracklet classifier (trees) and metrics**	Model development (4, shared), evaluation basics (shared)

Why Advik has the most important part: it is the part that decides whether the numbers can be believed (the evaluation protocol, the scoreboard, the held-out nights, fair model comparison), it contains the main new method (the deep search, +0.7 magnitude) and it connects all other parts into one running system, plus the live viewer used in the demo. If this part is wrong, every other part's numbers are meaningless. Advik also opens and closes the presentation.

How the parts connect (the "contract" between owners)

```
Aaryan  ->  units of images, split by night, cleaned (masks, noise, sky coordinates), blobs d
Paras   ->  (3x31x31 stamps + labels) -> 29 features + PCA -> XGBoost -> p(real) for every bl
Satvik  ->  tracks from the kept blobs -> 17 features -> XGBoost score -> known/unknown
Advik   ->  runs it all as one deployed path, measures it on planted fakes and held-out night
```

Handover	What is handed over	File to look at
Aaryan -> Paras	`load_exposure` output, the blobs table from `extract_sources` (x, y, flux, SNR, shape), the nights split, `configs/acquire.yaml`, `detect.yaml`	`clean/load_exposure.py`, `detect/extract_sources.py`, `acquire/`
Paras -> Satvik	`detections` table with a `score` column (cross-fitted for training nights) and the stage-1 threshold	`models/predict_realbogus.py`, `scripts/train_classical_bundle.py`
Satvik -> Advik	tracklets with `tracklet_score` and `known_name`	`pipeline/run_night_pipeline.py`
Advik -> all	the scoreboard: the one place numbers come from	`evaluate/build_scoreboard.py`, `reports/scoreboard.csv`

Rule: **every number in your slides must be traceable to a file** (a lesson document, a notebook output or `scoreboard.csv`). If you cannot point at the file, do not say the number.

Presentation order (adjust to your time slot; default 20 minutes + questions)

Order	Who	Time	Content
1	Advik	3 min	The problem, the idea, the whole pipeline in one picture, who does what
2	Aaryan	4 min	The data, difference imaging, preprocessing, splitting by night, leakage
3	Paras	5 min	Labels (injection-recovery), stamps, the 29 features + PCA, the model comparison (CNN as a reference), k-means junk clusters, the label-bug story

Order	Who	Time	Content
4	Satvik	5 min	Linking, the 17 features, trees and boosting, metrics, why accuracy fails
5	Advik	5 min	Evaluation protocol, scoreboard v1 to v4, deep search, **live demo** (3 min), limits, conclusion
	everyone		questions: each answers about their own part first; anyone may add

If you only get **12 minutes**: 2 + 2 + 3 + 3 + 2 (skip the live demo; show one screenshot per stage).

Timeline suggestion (about 2 weeks; compress if needed)

Days	Everyone	Advik	Aaryan	Paras	Satvik
1-2	Read `00_Start_Here` ; run the tests; open the viewer once	prepare the demo; check numbers against `scoreboard.csv`	read `02_Knowledge_Pack.md` ; re-run `data_acquisition` + `difference_imaging` notebooks	re-run `neural_networks` notebook; read `realbogus_cnn.md`	re-run `ml_b` `metrics_e` notebooks
3-6	Study your pack; write your own summary in your own words	scoreboard + deep search notebooks; draft the report's evaluation chapter	leakage audit notebook; EDA figures for the slides	dataset building + CNN notebooks; error-analysis story	linking + trac notebooks; r exercises
7-9	Slides from `03_Presentation_Guide.md` ; practise 3 times with a timer	integrate slides; rehearse the demo			
10-12	Cross-viva each other with `04_Viva_QA.md` and `06_Whole_Project_Viva_QA.md`				
13-14	Dress rehearsal, fix weak answers, finalise the report				

Rules for the team

1. **Everyone must be able to answer questions on the whole pipeline at the level of `06_Whole_Project_Viva_QA.md` , not only their own part.**
2. **Run things yourself.** Re-run your notebooks, change one setting, see what changes, and write what you saw. This is what makes the explanation yours (see `07_AI_Use_and_Integrity.md`).
3. **One source of numbers.** If a number changes, tell Advik; slides and the report are updated from one list.
4. **Say "I don't know" correctly:** "I did not test that; my guess is ... and I would check by ...". That is a good viva answer. A made-up number is not.
5. **Do not claim a discovery.** Ever.

Report structure (suggested; one section per owner)

1. Introduction and problem formulation (Aaryan) · 2. Related work and the paper(s) implemented (Aaryan + Paras) · 3. Data and preprocessing, exploration (Aaryan)
2. Labels and dataset (Paras) · 5. Models: the stage-1 classical model and its comparison (Paras), the tracklet models (Satvik) · 6. Evaluation methodology and results (Advik, with Satvik's metrics)
3. Deep search and live system (Advik) · 8. Limitations and negative results (Advik) · 9. Conclusion · References · AI-use declaration (everyone, see file 07)

06. Whole-project viva: questions every member must answer

How to use: take turns asking each other, answer out loud in 30-60 seconds, then read the model answer. The model answers are short on purpose; add your own example from *your* part.

The basics

Q1. What does the project do, in one sentence? It is an ML pipeline that searches archival ZTF telescope images for moving objects (asteroids), separates real ones from junk with two classical machine-learning models (gradient-boosted trees), checks them against the catalogue of known asteroids, and measures honestly how deep and how reliable the search is.

Q2. Why difference images? Subtracting an old reference of the same sky from tonight's picture cancels stars, galaxies and sky glow, so only new or moving things remain. That shrinks the search from about 1,950 sources to a few hundred strong ones per frame.

Q3. Why does the project need machine learning at all? Most blobs in a difference image are junk, in many shapes (dipoles, cosmic rays, edges, noise); only about 3% are real. Hand-written rules cannot capture all those shapes. So we measure 29 numbers per blob plus 20 PCA numbers and let XGBoost learn the shapes from labelled examples, and a second tree model combines many weak track-level clues better than a single cut. The rest of the pipeline (matched filtering, linking, shift-and-stack) is physics and search, not learning.

Q4. What are the two ML models, and what are their inputs and outputs? (1) XGBoost for blobs: input 49 numbers measured from a 3 x 31 x 31 stamp (29 hand-made features such as "looks like a star" and "already in the old picture", plus 20 PCA numbers), output p(real) for one blob. (2) XGBoost for tracks: input 17 numbers describing a track (speed, straightness, brightness steadiness, the first model's scores, forced photometry...), output a score for the whole track. A CNN was tried for (1) and tied (AP 0.898 vs 0.902); it is kept only as a comparison.

Q5. What is a unit and what is a tracklet? A unit is one field + CCD + quadrant + night. A tracklet is a set of detections in different frames that lie on a straight line at a plausible asteroid speed (at least 3 of 5 frames, within 1.5 px of the line, 8-120 arcsec/hour).

Data and honesty

Q6. Why split by night and not randomly? Frames from one night share the same stars, noise and artifacts. A random split would put near-copies in train and test and inflate scores. Splitting by night, decided in a config file before modelling, keeps the test honest.

Q7. Give an example of data leakage from your own project. Our first dataset had `mag` and `true_rate` columns that existed only for planted fakes, so "is mag missing?" equalled the label. Also, the random bogus tracklets were 222+ pixels off a line while real ones were within 2.6 px, so a one-line rule scored 100%. Trained on that data, a model got 50% (a coin flip) on realistic near-miss tracklets and 99% when trained on realistic data.

Q8. How do you get labels when the sky is not labelled? Three sources: planted fake asteroids (perfect labels, but only as real as the simulation), known asteroids from catalogues (real, but fewer, positions from JPL Horizons), and everything else called bogus (with a little label noise, because it includes undiscovered real objects).

Q9. How do you know your fakes look like real asteroids? A classifier trained to tell fakes from real known asteroids (matched in brightness, cross-validated by night) scored AUC 0.46-0.48, where 0.5 means indistinguishable. Also real asteroids and fakes both fade out between magnitudes 20.5 and 21.5.

Q10. What was the biggest bug and how was it found? The telescope pointing shifts between exposures (up to 12 px). We linked blobs in each frame's own pixel coordinates, so real asteroids did not line up while fakes (planted straight in pixel coordinates) did. Real asteroids all showed the same residual pattern, which pointed at a per-frame error. Fix: put every blob on frame 1's sky grid before linking; real residuals fell from about 2.5 px to 0.05-0.47 px. Lesson: test the simulation against real objects, not only against itself.

Models and metrics

Q11. Why not just report accuracy? Only about 3% of blobs are real, so "always bogus" scores 96.8% accuracy while finding nothing. We report recall (how many real objects we keep), precision (how pure the flagged set is), false alarms per frame and average precision.

Q12. Why is a precision-recall curve better than ROC here? ROC's x-axis divides by all bogus (65,000), so hundreds of false alarms look like a tiny step: ROC AUC was 0.95 while precision at useful recall was under 20%. Precision divides by the number of alerts, so it shows directly how polluted the output is.

Q13. How did you choose the thresholds? On the validation nights only, never on test: the stage-1 cut keeps 99% of real detections; the tracklet cut keeps 95% of real tracklets. Fixing recall first and then counting false alarms follows the cost argument: a missed asteroid is a lost discovery, a false alarm costs one minute of a person's time.

Q14. What is cross-fitting and why did you need it? The tracklet model uses the stage-1 scores as features. A model scoring nights it trained on is over-confident, and the tracklet model would learn that. So training nights are scored by models trained on other nights (4 folds of 2 nights), and validation/test nights by the final model. It is stacking without leakage.

Q15. How do you prevent overfitting? Stage-1 XGBoost: shallow trees (depth 4), 80% row and column subsampling, settings chosen on validation only; PCA keeps only 20 directions. Tracklet XGBoost: shallow trees (depth 3), small learning rate (0.05), 80% row and column subsampling. Everything is judged on nights the model never saw.

Q16. The tracklet classifier scores AP 0.999. Is it great? No. That number is saturated: 96% of tracklets in that data are real because we plant 300 fakes per night, the opposite of the real sky. It cannot show improvement. What matters is depth, known asteroids recovered, and bogus tracklets per night on held-out nights.

Results and evaluation

Q17. What does "50% completeness at magnitude 20.5" mean? Plant fake asteroids of known brightness; at magnitude 20.5 the system recovers half of them. Bigger magnitude is fainter, so a deeper system has a bigger number.

Q18. How did the system improve from v1 to v4, and why? v1 reached depth about 20.5. We measured where asteroids were lost: at linking, not at the classifier. Relaxing linking found 11% more but created about 1,500 chance tracks per night, which became hard negatives; adding two physics features (static neighbours, forced photometry) controlled them. Depth on the test night rose to 21.4. A multi-frame CNN only tied the simple XGBoost, so we kept XGBoost.

Q19. Why did 3x more training data (v5) not help? At equal false-alarm levels v4 was equal or slightly better. The system is limited by how many photons one 30-second frame holds, so more examples cannot reveal what is not in the image. That is why the deep search, which adds frames, gained +0.7 magnitude, more than any model change.

Q20. What is the deep search and why does it work? Shift the frames along a trial motion so a moving object lands on the same pixel and add them. The signal adds N times but the noise only sqrt(N) times, so 8 frames gain about 2.8x, +1.1 magnitudes. We try 4,888 motions on the GPU. Because we pick the best of thousands of trials, noise alone reaches 5 sigma (look-elsewhere effect), so we require 6 sigma, and a null test with shuffled frame times produced 0 false tracks.

Q21. How do you know the deep search is not finding noise? Calibrate the threshold on pure noise; a null test (shuffle the frame times so real movers no longer line up) gives 0 false tracks after the velocity-consistency cut; the cut also kept 20 of 20 known asteroids. Injection-recovery gives 94-100% recovery at magnitude 21.4-21.9.

Q22. Why compare models at the same false-alarm level? A rule like "95% recall on validation" lands on different points of each model's trade-off curve. v5 looked 0.8 mag worse at its own threshold, but at matched alarms it tied v4. Fair comparison means the same operating point.

Limits and ethics

Q23. Did you discover an asteroid? No. The system recovers known asteroids reliably to about magnitude 21. One faint object in a related two-night study passes every check we have (an orbit fit, noise tests, no catalogue match), but that is not a discovery: it needs a human to read the stamps and the Minor Planet Center's process. Nothing has been reported.

Q24. What are the main limitations? Depth is capped by single frames (the deep search helps on one quadrant of one night); the training data covers one field mostly; "real" labels come partly from simulation; catalogue positions have errors; the test night is no longer perfectly untouched because real bugs were fixed after looking (so a separate holdout night exists); and "unknown" only means "not in the catalogue we queried".

Q25. What would you do with more time? Evaluate on more fields and nights; use the k-means junk clusters to build targeted features; add a learning curve; run the deep search on more nights; get an independent confirmation from another survey.

Q26. How reproducible is it? Configs hold every setting; the night split is in a config; random seeds are `[project seed, night date]`; model versions are bundles with the commit and nights used; archive answers and known-asteroid lookups are cached; about 150 offline tests; `environment.yml` pins the software.

Q27. How did you use AI in this project? (answer truthfully; see `07_AI_Use_and_Integrity.md`) The project was developed with an AI coding assistant (Claude) under Advik's direction, including code, documentation and diagrams; each of us then re-ran and studied our part. We declare this as the course policy requires. Be ready to show that you understand your own part without notes.

Where the course topics are

Q28. Which syllabus topics does the project use? Logistic regression (baseline), feature engineering, SVM, decision trees, bagging, random forests and boosting (XGBoost, deployed), PCA, k-means clustering, regularisation, train/validation/test split by night, cross-validation by night, error analysis and skewed-class metrics, ROC vs precision-recall. Neural networks (from-scratch notebook, CNN) only as a comparison. Not used: normal equation, hierarchical clustering (we say so openly).

Q29. If I gave you 3 minutes and a whiteboard, how would you draw the pipeline? Data (IRSA, split by night) -> difference image -> detect blobs -> stamps -> 29 features + PCA -> XGBoost -> keep blobs above a cut -> link into tracks on the sky grid -> 17 features -> XGBoost -> cross-match with known asteroids -> unknown candidates -> a person. Underneath: planted fakes and held-out nights measure every stage; the deep search adds frames along trial motions for fainter objects.

Q30. What did you personally learn that surprised you? (prepare your own answer) Good examples: a model's most confident "mistakes" revealed a labelling error; a near-perfect AP meant nothing; adding data did not help; the evaluation design mattered more than the model.

07. AI use and academic integrity (please read this before you present)

What the course policy says

From the *Course Policy*, section "GenAI Usage" (Projects):

- **Allowed:** use AI to "assist in the planning, design, and outlining of project ideas" and to "conduct research, analyze data, and find relevant information". "GenAI is permitted to be used only for the deployment of model or learning new concepts beyond classroom. Students need to cite the references used."
- **Not allowed:** "Develop entire project; the core implementation and critical thinking aspects of the project must be the students' own work." And: "Use AI in ways that bypass the learning process and objectives of the course."
- General rule for assignments: do not use AI "for creating entire code or solutions using AI without understanding and personal contribution" and do not copy "AI-generated content and pass it off as their own work".
- AI tools are **not allowed during exams and quizzes**.

The honest situation

This project's code, lessons, tests and diagrams were developed in a long working session between **Advik and an AI coding assistant (Claude)**. Advik directed the work, made decisions and reviewed results; the assistant wrote most of the code and documents. The folder you are reading, including the viva answers, was also generated with AI help.

That does not make the project worthless, but it does mean two things you must handle honestly:

1. **The policy expects the core implementation and critical thinking to be yours.** Be ready to show it: you must understand every line of your part, and ideally have done real work in it yourself (re-run, modify, extend, write in your own words).
2. **You must declare the AI help.** Hiding it, if discovered, is a much bigger problem than the help itself.

What to do (practical, in order)

1. **Tell the coordinator now**, in a short email, how AI was used and ask whether the project is acceptable as it stands and what they expect from you personally. Do this before the presentation, not after a question in the viva.
2. **Write the declaration** in the report (draft below, edit so it is true for you).
3. **Do real work on your own part**, so that the declaration is honest and the viva is easy:
   - re-run your notebooks from the first cell and write down what each output means in your own words;
   - change one thing (a threshold, a layer, a feature, a split) and record what happened and why;
   - add one small experiment of your own (the optional extras in `04_Course_Mapping.md` are good candidates);
   - write your report section yourself; use `02_Knowledge_Pack.md` to learn, not to copy.
4. **Cite sources.** Papers are in `04_Course_Mapping.md`; also cite software (PyTorch, XGBoost, scikit-learn, astropy, SEP) and data (ZTF via IRSA, with the acknowledgement text from IRSA's release notes, https://irsa.ipac.caltech.edu/data/ZTF/docs/releases/ztf_release_notes_latest).
5. **In the viva, do not bluff.** "That part is Paras's; from my understanding it works like this..." is fine.

Draft declaration (edit to match what is true for you)

> **Use of generative AI.** This project was developed with the assistance of an AI coding assistant (Claude, by Anthropic), used under the direction of Advik Divekar. The assistant helped to plan the project, write

> and test the source code, write the lesson documents and diagrams, and prepare study material. Each team member then studied, re-ran and [describe what you personally did: e.g. modified and evaluated] the part they present: Advik (system, evaluation and deep search), Aaryan (data and preprocessing), Paras (labels and the neural network), Satvik (linking, tree models and metrics). We understand and can explain the methods and results, and all references used are cited. [Add anything else that is true: experiments you added, data you inspected yourself, errors you found.]

A note about the project itself

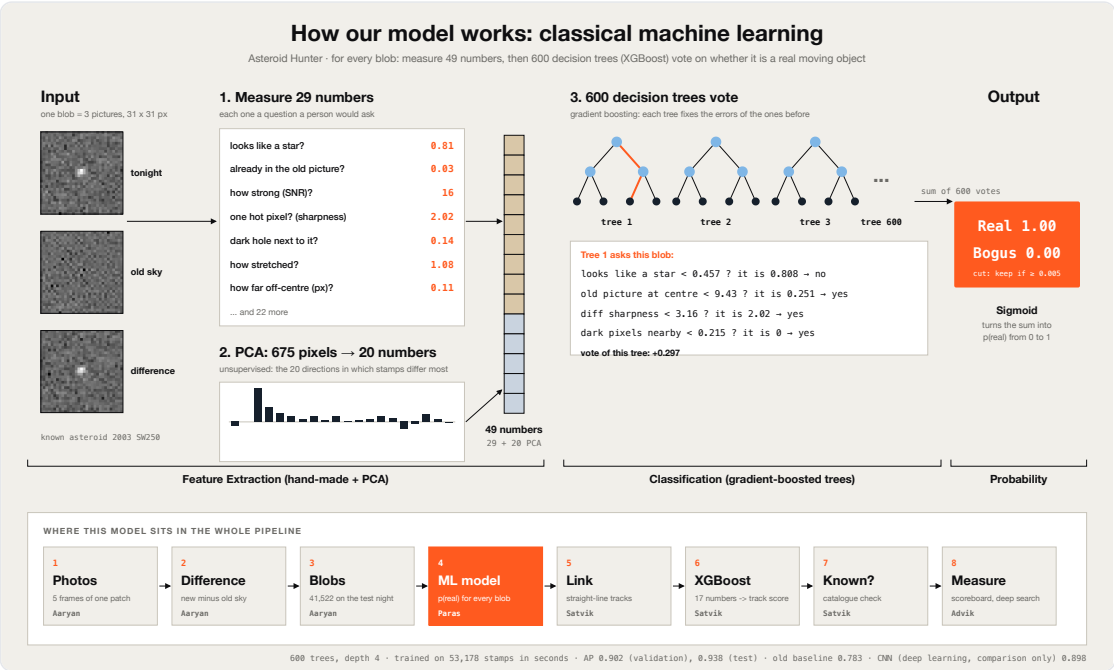
Because the policy is about *learning*, the most valuable thing you can do with this folder is use it the way a student uses a worked example: read, run, break it on purpose, explain it out loud, and then extend it.

08. Glossary

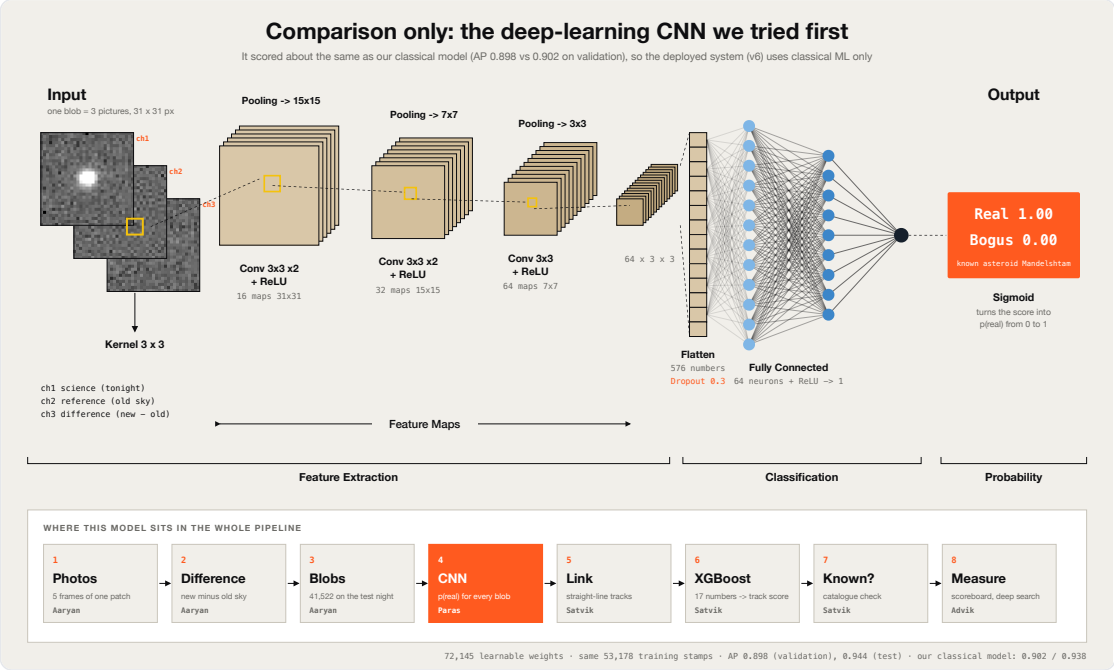
Term	Meaning (plain words)
Accuracy	fraction of all predictions that are right; misleading when one class is rare
Adam	a gradient-descent variant that adapts each weight's step size; our optimiser
Adversarial validation	train a classifier to tell two datasets apart; if it cannot (AUC 0.5) they are alike
Anomaly detection	finding items that do not fit the known pattern; our "unknown" tracks are anomalies by catalogue lookup
Arcsecond ("")	1/3600 of a degree; one ZTF pixel is about 1.01 arcsec
Asteroid (main belt)	rocky body orbiting the Sun, mostly between Mars and Jupiter
Augmentation	making extra training examples by rotating/mirroring; sky has no "up"
Average precision (AP)	area under the precision-recall curve; a single summary for rare positives
Backpropagation	chain rule from output back to every weight; gives all gradients in one sweep
Bagging	train many models on random samples and average them (random forest)
Base rate	how common the positive class really is (about 3% real)
Bias / variance	systematic error vs sensitivity to the training sample; underfit vs overfit
Bleed trail	charge from a saturated star spilling along a CCD column; causes false fast tracks
Boosting	trees built one after another, each fixing the previous errors (XGBoost)
Bogus	a detection that is not a real moving object (artifact, noise...)
Bundle (model)	a saved version: stage-1 model + tracklet model + thresholds + provenance (`models/v6` is deployed: classical ML; `models/v4` has the CNN)
Calibration	making scores mean real probabilities (reliability plot, isotonic regression)
CCD / quadrant	a camera chip / one quarter of a chip (about 3072 x 3080 px)
Class imbalance	one class is much rarer than the other
CNN	convolutional neural network: filters slide over an image to find local patterns (we keep it only as a comparison)
Feature engineering	measuring numbers with a meaning from raw data (29 per stamp here)
PCA	principal component analysis: keeps the few directions in which the data vary most (675 pixels -> 20 numbers)
k-means	clustering: groups points around k centres, without labels (6 kinds of junk)
SVM	support vector machine: the widest-margin boundary between classes
Common grid	frame 1's pixel grid; every blob is projected onto it before linking
Completeness	fraction of planted fakes recovered (recall by brightness)
Confusion matrix	counts of true/false positives and negatives
Cross-fitting	scoring each fold with a model that did not train on it, so stage 2 sees honest scores
Cross-validation (GroupKFold)	repeated train/test by groups (nights), never by random rows
Deep search	shift-and-stack along trial motions to reach fainter than one frame
Depth	the brightness (magnitude) where 50% of fakes are recovered
Difference image	science minus reference: only new things remain
Dipole	bright half beside dark half: imperfect subtraction of a bright star
Dropout	randomly switching off neurons during training to reduce overfitting
Early stopping	stop training when validation performance stops improving; keep the best epoch
Ephemeris	a table of where an object should be at given times
FITS	the standard astronomical image file format
Forced photometry	measuring flux at a predicted position even where nothing was detected
Gradient descent	repeatedly move weights against the gradient of the loss

Term	Meaning (plain words)
Holdout night	a night never used for choosing anything; opened only to write one number
IRSA	NASA/IPAC archive that serves the ZTF data
Injection-recovery	plant fakes, run the pipeline, count what comes out
Isotonic regression	monotone curve used to calibrate scores
KD-tree	data structure for fast nearest-neighbour lookups
Leakage	information at training time that will not exist at use time (target, group, shortcut, shift)
Linking	joining blobs from different frames into one straight-line track
Log-loss (cross-entropy)	the loss for probabilities: confident and wrong costs a lot
Look-elsewhere effect	testing many places makes chance peaks likely; raise the threshold
Magnitude	log brightness scale: bigger is fainter; 5 mag = 100x
MAD	median absolute deviation; robust spread: sigma = 1.4826 x MAD
Mask	per-pixel flags (saturated, dead, cosmic ray, ghost...)
Matched filter	smoothing with the expected source shape to lift faint sources out of noise
MPC	Minor Planet Center: the official keeper of asteroid observations
Normalisation	putting stamps in units of their own noise: (x - median) / (1.4826 MAD)
Null test	repeat the search on data where there is nothing to find (shuffled times) to measure false alarms
Opposition	asteroid opposite the Sun: brightest, most numerous
Overfitting	learning the noise of the training set
Precision / recall / FPR	purity of what is flagged / share of real objects kept / share of bogus wrongly flagged
PSF	point spread function: the blur shape of a point source
Reference image	deep picture of the same sky built from old exposures
Regularisation	any device that discourages overfitting (L2, dropout, early stopping, subsampling)
ROC / PR curve	recall vs false-positive rate / precision vs recall
Scoreboard	one function that computes every headline number from the deployed pipeline
Sigma	the typical size of the noise; "5 sigma" = 5 times it
SkyBoT / JPL Horizons	services that predict where known asteroids are (Horizons is more accurate)
SNR	signal-to-noise ratio: flux divided by its uncertainty
Stamp (triplet)	3 x 31 x 31 cut-out: science, reference, difference
Shift-and-stack	shift frames to follow a trial motion, then add them
Tracklet	a set of detections that line up as one moving object
Unit	one field + CCD + quadrant + night
WCS	the mapping between pixels and sky coordinates (RA, Dec)
XGBoost	a fast gradient-boosted tree library
ZTF	Zwicky Transient Facility, a survey telescope at Palomar Observatory

Diagrams

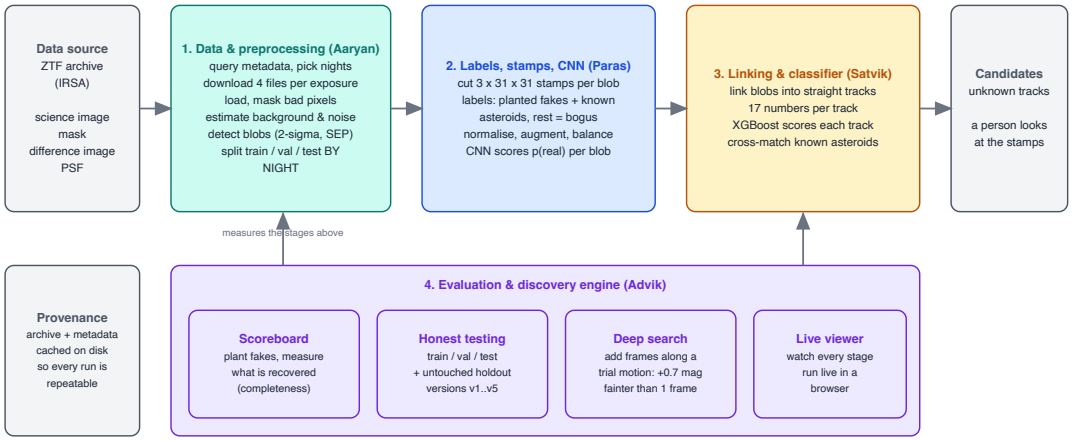


00 How Our Model Works



00b CNN Comparison

The whole system: from telescope images to candidate asteroids



Two ML models do the learning: a CNN on pixels (stage 2) and gradient-boosted trees on track numbers (stage 3).
Everything else is physics, search and statistics. Nothing is reported as a discovery without a person checking it.

Colour = owner of that part of the work.

01 system overview

Splitting by NIGHT, decided before any modelling

Every detection, fake and stamp from one night lives in exactly one group (configs/acquire.yaml).

TRAIN (8 nights): the models learn from these

2020-09-19	2020-09-21	2020-09-23	2021-09-11	2022-09-22	2022-09-24	2023-09-20	2023-09-25

VALIDATION (2 nights): choose settings and thresholds here

2021-09-12	2023-09-15

TEST (1 night): opened once for the final number

2019-02-08

the best night: sharp seeing, field at opposition, the most asteroids

HOLDOUT (1 night): never touched

2022-06-17

Why by night? Frames of one night share stars, noise and artifacts. Random row splits would leak that into the test.

02 split by night